

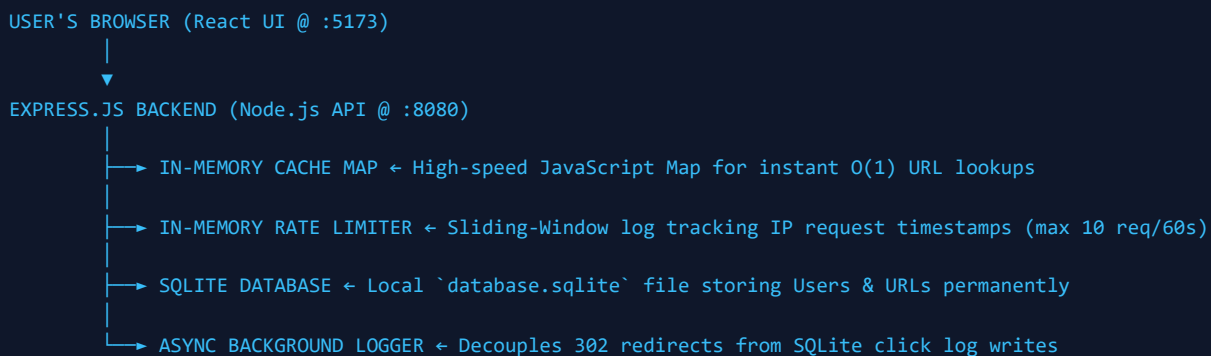
SwiftURL — The Complete Project Guide (Zero-Install Edition)

Node.js, Express, SQLite, React & In-Memory Architecture | Engineered by Koushik Bobba

1. WHAT IT DOES: The Core Concept

SwiftURL takes long, messy web links (e.g., `https://amazon.com/category/laptop/...`) and converts them into short, clean, 6-character links (e.g., `http://localhost:8080/r/q0V`). When a user clicks the short link, they are instantly redirected to the original long URL. Simultaneously, the system tracks analytics—recording the user's IP, browser (User-Agent), and timestamp—without slowing down the redirect experience.

2. HOW IT WORKS: The End-to-End Pipeline



3. TECHNOLOGY STACK: What We Used & Why

Node.js & Express.js

Backend API

What: Handles HTTP routing, JWT token verification, Base62 URL shortening, and analytics queries.

Why: Provides excellent asynchronous non-blocking I/O performance. Running without Docker makes it lightweight and runnable on any machine with just `node index.js`.

SQLite

Zero-Install Database

What: A lightweight, file-based relational database (`database.sqlite`).

Why: Eliminates heavy database installations (like PostgreSQL/Docker) while maintaining full SQL query capabilities, foreign key constraints (`ON DELETE CASCADE`), and ACID compliance.

In-Memory Cache & Rate Limiter High Speed Protection

What: Native JavaScript `Map()` objects acting as an in-memory key-value cache and rate limit tracker.

Why Cache: Reduces database file reads. Lookups from JavaScript memory take ~0.01ms vs ~5ms for disk reads.

Why Rate Limiter: Implements the **Sliding Window Log** algorithm. Tracks request timestamps per IP and blocks requests if count > 10 in a 60-second window.

React + Vite Frontend Dashboard

What: Single Page Application (SPA) dashboard with dynamic state management.

Why: Uses `Recharts` for click throughput charts, `lucide-react` for icons, and `localStorage` for JWT session persistence so the user stays logged in across page reloads.

4. KEY ENGINEERING ALGORITHMS

Base62 Encoding for URL Generation:

Instead of generating random short codes (which suffer from collision issues), we use SQLite auto-incrementing row IDs (1, 2, 3...). We convert that integer into a Base62 string (0-9, a-z, A-Z). Since every row ID is unique, every short code is mathematically guaranteed to be 100% collision-free!

5. ISSUES FACED & HOW WE FIXED THEM

Issue 1: Docker Desktop & WSL Corruption on Windows

Problem: Docker failed to start due to `Wsl/CallMsi/Install/REGDB_E_CLASSNOTREG` error in Windows.

Solution: We migrated the entire architecture away from Docker/Postgres/Redis into a zero-install Node.js + SQLite setup, allowing the app to run natively with zero virtual machine overhead.

Issue 2: JWT Token Missing Headers & User State Loss

Problem: The frontend was getting "Missing or invalid token" errors because API requests were sent without the `Authorization: Bearer` header, and page reloads reset the navbar user state.

Solution: Attached the Bearer token to all `fetch()` calls in `ShortenerCard.jsx` and `App.jsx`, and saved `user_info` in `localStorage` to persist login state across reloads.

6. TOP INTERVIEW QUESTIONS & ANSWERS

Q1: How does your Sliding Window Rate Limiter work without Redis?

I implemented the algorithm using an in-memory JavaScript `Map`. Each client IP maps to an array of request timestamps. On every request, we filter out timestamps older than 60 seconds (`now - 60000`) and count the remaining items. If the count exceeds 10, we return HTTP 429.

Q2: Why use SQLite instead of a traditional DB server?

SQLite is serverless and self-contained in a single file (`database.sqlite`). For applications with low-to-medium write concurrency or zero-config deployment needs, SQLite offers zero network overhead (queries execute in-process) and handles thousands of reads per second effortlessly.

Q3: How do you handle asynchronous analytics logging without a queue?

We use Node.js non-blocking event loop using `setTimeout(..., 0)`. When a user requests a redirect, the server returns the HTTP 302 response immediately. The analytics DB write is scheduled on the next tick of the event loop, ensuring the user never waits for disk I/O.